

• Lava-Me

SaaS multi-tenant para gestão de lava-jatos e estéticas automotivas — conectando gestores, funcionários e clientes em uma única API.

APRESENTAÇÃO TÉCNICA DO PROJETO

Lucas José Heitor Fernandes · líder Letícia Lopes Wanderson Mello
Maurício Monteiro Marcos Barbosa

Professor
Prof. Edeilson Milhomem da Silva

Sumário

01 Contexto e Proposta de Valor

02 Arquitetura e Domínio

03 Regras de Negócio e Multi-tenant

04 Processo, Sprints e Releases

05 Testes e Manutenibilidade

06 IA • Twelve-Factor App

07 Entrega e Avaliação

08 Equipe e Aprendizagem

Contexto e Proposta de Valor

O PROBLEMA

A operação de lava-jatos vive em planilhas, papel e WhatsApp. Faltam rastreio do estado de cada serviço, registro fotográfico de vistoria e visibilidade para o cliente final.

A PROPOSTA

Uma plataforma SaaS multi-tenant com a Ordem de Serviço rastreável de ponta a ponta — vistoria fotográfica obrigatória, agendamento e avaliação — servindo gestores, funcionários e clientes pela mesma API.

Gestor

Portal web · controle operacional

Funcionário

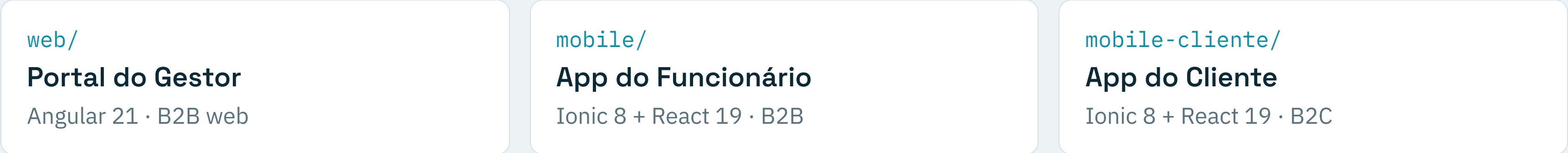
App mobile · execução da OS

Cliente

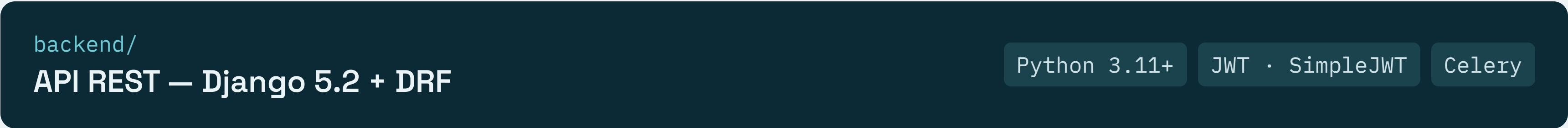
App mobile · agendamento e avaliação

Arquitetura em 3 Camadas

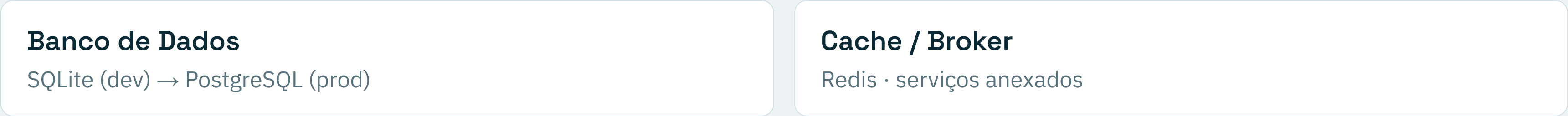
CLIENTES



↓ REST · JSON · JWT ↓



↓



Domínio Central: a Ordem de Serviço



↘ BLOQUEADO_INCIDENTE

Desvio a partir de **EM_EXECUCAO**. Somente o **Gestor** desbloqueia, e o desbloqueio exige uma nota justificando.

↘ CANCELADO

Transição permitida **somente a partir de PATIO** — depois que a execução começa, a OS não pode mais ser cancelada.

Regras de Negócio e Multi-tenant

REGRAS CENTRAIS

5 **fotos obrigatórias** na vistoria geral e na finalização da OS.

1 Um funcionário **não pode ter 2 OS em EM_EXECUCAO** simultaneamente.

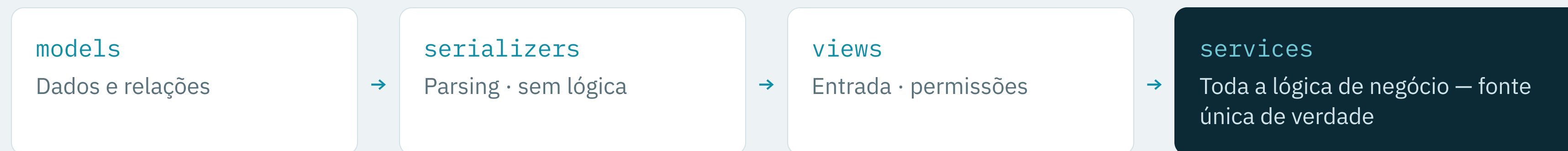
! **BLOQUEADO_INCIDENTE** exige nota do Gestor para desbloquear.

ISOLAMENTO MULTI-TENANT

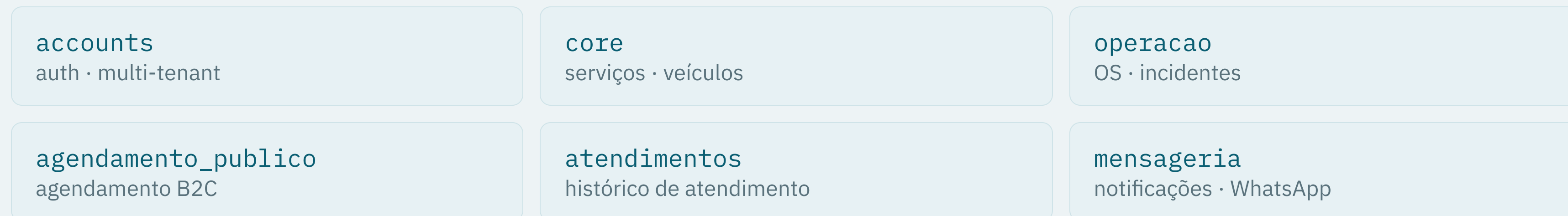
Todo *queryset* é filtrado pelo **estabelecimento** do usuário logado, via `perfil_gestor / perfil_funcionario`.

Clientes são isolados por **FK em Veiculo** — sem vazamento de dados entre tenants.

Camadas do Backend e Organização em Apps



APPS DJANGO



Processo de Desenvolvimento

01

Scrum ágil

Sprints de 1–2 semanas, com cerimônias e entregas incrementais por iteração.

02

GitFlow

`main · develop · feature/* · release/* · hotfix/*`

03

Documentation-First

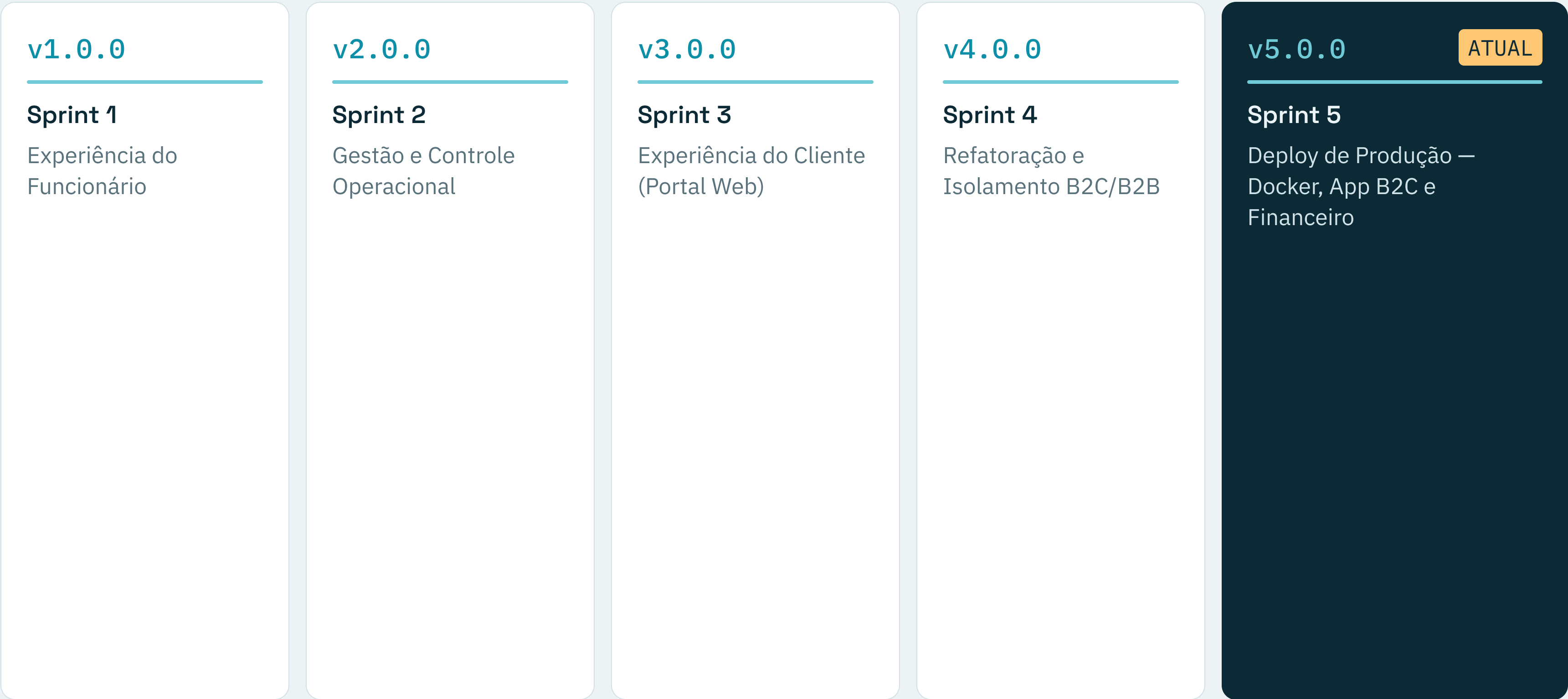
Cada funcionalidade nasce como RF em `docs/3_regras_negocio/`, validado antes de implementar.

04

TDD

Testes da seção 2 do RF escritos e falhando **(Red)** antes do código **(Green)**.

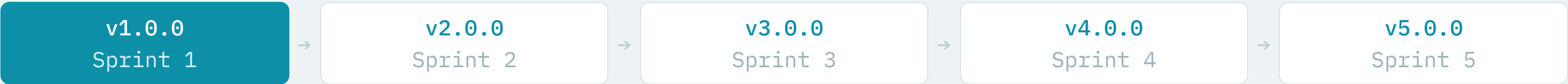
Linha do Tempo: Sprints e Releases



Sprint 1 — Fundação e Experiência do Funcionário

release v1.0.0

Objetivo — fundar o sistema, alinhar a linguagem de domínio e garantir que o operador registre o ciclo de vida da OS com segurança.



- RF-01 Cadastro de Usuário
- RF-02 Login de Usuário (JWT)
- RF-03 Refatoração Ubíqua → OrdemServico
- RF-04 Visualizar Pátio
- RF-05 OS em Abas Progressivas

- RF-06 Vistoria Inicial e Avarias — 5 fotos
- RF-07 Liberação / Check-out
- RF-08 Entrada Avulsa (check-in expresso)
- RF-09 Registro de Incidente
- RF-10 Sincronização em Background

Regras críticas — bloqueio de OS com incidente · imutabilidade da vistoria inicial após avanço · 5 fotos obrigatórias.

Sprint 2 — Gestão e Controle Operacional

release v2.0.0

Objetivo — configuração da unidade e equipe, visibilidade operacional em tempo real (Kanban) e histórico com auditoria.



TRILHA 1 · PARAMETRIZAÇÃO

RF-11 Gestão de Serviços (CRUD + soft delete)

RF-12 Administração de Funcionários

RF-13 Configurações da Unidade

TRILHA 2 · TEMPO REAL

RF-14 Kanban de Pista ✓

RF-15 Central de Incidentes Pendentes ✓

RF-16 Auditoria e Desbloqueio ✓

TRILHA 3 · HISTÓRICO

RF-17 Histórico Consolidado de Atendimentos

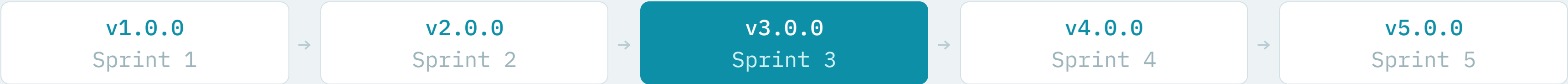
RF-18 Auditoria de Qualidade Visual

RNFs em destaque — isolamento multi-tenant estrito (anti-IDOR) · proibição de hard delete em registros vinculados a OS · resolução de incidente atômica e transacional. ✓ implementado e validado.

Sprint 3 — Experiência do Cliente / Portal Web

release v3.0.0

Objetivo — abrir o sistema ao cliente final via portal público de autoagendamento e dar ao gestor visão executiva do negócio.



- RF-19/20 Dashboard Executivo (faturamento, volume, eficiência)
- RF-21 Autoagendamento B2C — “fricção zero”
- RF-22 Grace Period de Horário (5 min)
- RF-23 Integridade de Agendamento (SELECT FOR UPDATE)

- RF-24 Cancelamento Autônomo (slug UUID por WhatsApp)
- RF-25 Painel do Cliente Multicentralizado
- RF-26 Transparência Pós-Venda (galeria pública)

Privacidade — isolamento por titularidade (cliente só vê seus veículos/OS) · dados financeiros internos do estabelecimento nunca expostos ao cliente.

Sprint 4 — Refatoração e Isolamento B2C/B2B

release v4.0.0

Objetivo — consolidar a arquitetura para 3 frontends seguros e independentes e iniciar o app B2C dedicado (mobile-cliente).



RF-27 Core e Banco de Dados
Refatoração de modelos para separar B2C/B2B (resolve DT-010).

RF-30 Refatoração de UX Operacional B2B
Otimização do fluxo do funcionário — pulo automático de etapas.

RF-28 Portal do Cliente: Arquitetura e Mapa
Base do mobile-cliente e mapa de descoberta de estabelecimentos.

RF-31 Unificação de APIs e Segurança
Endpoint único GET /api/shared/historico/ — fecha brechas de IDOR.

RF-29 Jornada e Checkout (web + mobile)
Agendamento completo: veículo, serviço e horário no app B2C.

RF-32 Painel Financeiro Básico (Web)
Relatório de faturamento com filtros.

Sprint 5 — Deploy de Produção

release v5.0.0

MAIS RECENTE

Objetivo — levar o ecossistema a produção com infraestrutura dockerizada e concluir o app B2C.



Testes e Manutenibilidade

TESTES POR CAMADA

Backend

`pytest` — centenas de testes, cobertura HTML em `htmlcov/`, `--reuse-db`

Web — Angular

`Vitest` (25 arquivos `.spec.ts`) + `Cypress` E2E

Mobile — Ionic / React

`Vitest` + `Cypress`

MANUTENIBILIDADE

Separação estrita em camadas

Os `services` concentram as regras — uma fonte de verdade.

Soft delete

`is_active=False` — nunca hard delete com OS vinculadas.

Design system de tokens CSS

No mobile, sem CSS inline — estilos por tokens.

Infraestrutura de IA: RAG + MCP

RAG

Base vetorial

ChromaDB local indexa regras de negócio, glossário e a documentação em `docs/` para decisões contextuais.

MCP

Contexto em tempo real

`mcp_server.py` expõe schema do banco, logs e padrões de UI aos agentes.

AGENTES

Agentes autônomos

Atuam em code review, auditorias de segurança e implementação seguindo workflows documentados.

`write_feature``review_pr``audit_isolation``audit_frontend``audit_backend`

The Twelve-Factor App — 12 fatores, evidências reais

01 Codebase

monorepo Git · GitFlow · 5 releases (v1-v5)

02 Dependencies

requirements*.txt · package.json · venv isolado

03 Config

python-dotenv · settings.py lê os.environ

04 Backing services

Postgres · Redis · Evolution API via Docker/env

05 Build, release, run

make dev · make release v=X.Y.Z · npm run build

06 Processes

API stateless · estado em JWT e no banco, não na memória

07 Port binding

Django :8000 · Angular :4200 · Ionic :5173/8100

08 Concurrency

Celery — tarefas assíncronas fora do web

09 Disposability

Containers Docker · workers Celery — start/stop rápido

10 Dev/prod parity

.env.example documenta a paridade de config

11 Logs

dictConfig → console + logs/dev.log

12 Admin processes

manage.py migrate · seed_data · createsuperuser

Planejado vs. Entregue

100%

do escopo planejado entregue. Sprints 1–5 concluídas e tagueadas (v1 → v5), incluindo o deploy de produção.

DÉBITOS TÉCNICOS – CONHECIDOS E RASTREADOS

DT-002 Rate limiting do DRF usa cache em memória — não compartilhado entre workers.

DT-003 SQLite em dev; migração para PostgreSQL planejada para produção.

DT-004/5 Melhorias de UX pendentes — auto-preenchimento e feedback de cancelamento.

documentados em [docs/_privado/DIVIDAS_TECNICAS.md](#)

Avaliação do Produto Entregue

A proposta de valor foi atingida?

Sim. O ciclo completo da OS — vistoria fotográfica, execução, liberação e finalização — opera com multi-tenant e os três frontends integrados à mesma API.

Está pronto para uso?

Sim, como MVP operacional. Empacotado para produção via Docker (Postgres + Redis + Nginx) e APK via Capacitor; evolução contínua planejada.

Cobertura de testes sustenta as regras críticas

Isolamento por tenant validado em auditoria

NPS por estrelas captura feedback real

Critérios de Produto: do MVP às Métricas

MVP

Núcleo operacional da OS entregue e usável de ponta a ponta.

Validação de requisitos

Cada RF documentado e validado antes de implementar (Doc-First).

Inovação

Vistoria fotográfica + máquina de estados + infra de IA vs. planilhas e WhatsApp.

Escalabilidade

API stateless, multi-tenant, Celery/Redis, Postgres e Docker.

Feedback & Métricas

NPS por estrelas; tempo em cada estado, OS finalizadas e avaliação média.

Iteração

Ciclo de sprints com releases tagueadas e débitos técnicos rastreados.

Equipe e Divisão de Trabalho

Heitor Fernandes

@HeitorFernandes04

Líder do projeto · full-stack

Lucas José

@yamatosz

Desenvolvedor full-stack

Letícia Lopes

@LeticiaGLopes-151

Desenvolvedora full-stack

Wanderson Mello

@WandersonAMello

Desenvolvedor full-stack

Maurício Monteiro

@MontDeP

Desenvolvedor full-stack

Marcos Barbosa

@eziors

Desenvolvedor full-stack

Divisão por demanda de cada sprint — todos atuaram em back-end e front-end.

Aprendizagem Baseada em Projetos

O QUE EXERCITAMOS

Processo ágil na prática — sprints, cerimônias e entregas incrementais.

Auto-gestão — autonomia para planejar e dividir o trabalho.

Avaliação de pares — code review e feedback contínuo entre o time.

O QUE EVOLUIU NA EQUIPE

↑ Comunicação

↑ Trabalho com prazos

↑ Coesão da equipe

Sugestões para a Disciplina

Projeto real

Construir um produto de ponta a ponta consolidou o aprendizado.

Autonomia

Liberdade para decidir arquitetura, stack e processo.

Práticas de mercado

Ágil, testes, CI e deploy aproximaram a turma da realidade.

Saldo positivo — sem sugestões de mudanças significativas no formato da disciplina.

Obrigado.

Perguntas?

Lucas José Heitor Fernandes · líder Letícia Lopes Wanderson Mello Maurício Monteiro
Marcos Barbosa

Professor
Prof. Edeilson Milhomem da Silva